

Audyt projektu

Politechnika Monopoly

Analiza zgodności z wymaganiami projekty-2025.pdf
Struktura bazy danych · martwy kod · wydajność · rekomendacje

Repozytorium: javaprojekt (Spring Boot 3.5, Java 17, PostgreSQL, Thymeleaf, WebSocket, Three.js)

Data audytu: czerwiec 2026

Dokument przygotowany na potrzeby prezentacji i dalszego rozwoju projektu.

1. Wprowadzenie

Projekt Politechnika Monopoly to rozbudowana aplikacja WWW wykraczająca poza typowy zakres wymagań z projektu-2025.pdf: pełna gra wieloosobowa, ranking ELO, koło fortuny, lootboxy, znajomi, moderacja i plansza 3D. Niniejszy audyt ocenia zgodność z punktacją egzaminacyjną, identyfikuje martwy kod oraz proponuje usprawnienia architektury — w szczególności przeniesienie aktywnej rozgrywki z bazy danych do pamięci serwera (RAM).

2. Zgodność z wymaganiami (projekt-2025.pdf)

Legenda: TAK = spełnione | CZĘŚCIOWO | NIE = brak lub sprzeczne z kodem

Wymaganie	Status	Uwagi
≥3 klasy domen, kompozycje, Date, walidacja	TAK	User, GameSession, Property itd.
Logowanie, ≥3 role (admin)	TAK	ADMIN, MODERATOR, USER, GUEST
Thymeleaf + WCAG 2.1	CZĘŚCIOWO	Dobre na stronach statycznych; plansza 3D słabsza
CRUD MVC i REST dla klas domen	CZĘŚCIOWO	Brak PropertyController — usunięty
Walidacja 6 reguł formularza	TAK	RegistrationForm + PasswordMatches
Edycja na danych bieżących	TAK	SettingsController
Współdzielenie danych / link publiczny	TAK	/u/{username}, zaproszenia do gry
Sortowanie 3 kryteria × 2 kierunki	NIE	Było w Property CRUD — brak po usunięciu
Zapamiętanie sortowania (cookies)	NIE	Brak cookies sortowania; motyw w localStorage
Filtrowanie: data + pole domen	NIE	Brak widoku z filtrem
Zapis do DB przy wylogowaniu/sesji	NIE	Każdy ruch zapisuje do PostgreSQL od razu
Rejestracja, strona powitalna	TAK	register.html, index.html
Lista użytkowników + role (admin)	TAK	AdminController
Usługa REST	TAK	~24 endpointy gry + ranking
Klient REST (serwer → API)	NIE	Brak RestTemplate/WebClient w Javie
Spring Security z bazą	TAK	CustomUserDetailsService, BCrypt

Szacunkowo projekt zdobywa ok. 53–57 pkt z ~56 pkt wymaganych, lecz prowadzący może odejmować punkty za brak Property CRUD, sortowania/filtrowania, zapisu sesyjnego oraz nieaktualną dokumentację (docs/MAPA-WYMAGAN.md opisuje pliki, których już nie ma).

3. Mocne strony projektu (ponad wymagania)

- Pełny silnik Monopoly: kupno, czynsz, ulepszenia, karty szansy, bankructwo, boty.
- Synchronizacja w czasie rzeczywistym przez WebSocket (STOMP).
- Metagame: ELO, poziomy, koło fortuny, skrzynki, sklep, historia meczów.
- System znajomych, zaproszeń do gry i publiczne profile graczy.
- Moderacja: weryfikacja legitymacji, zawieszenia kont, ostrzeżenia.
- Panel administratora z podglądem aktywnych sesji i zarządzaniem użytkownikami.
- Plansza 3D (Three.js) - estetyka Business Tour, bez konieczności upraszczania do 2D.

4. Martwy i nieużywany kod

Backend — encje bez realnego użycia w runtime:

- Property + PropertyRepository — encja pod CRUD egzaminacyjny; gra używa `GamePlayer.ownedPositions`.
- Achievement, GameLog — kompozycja w User, ale nigdy nie zapisywane w kodzie.
- BoardTile, BoardCard, Rank, DailyTask — seed w DataInitializer; silnik czyta GameEconomy (kod).
- db/add-state-epoch.sql — kolumna state_epoch bez pola JPA w GameSession.

Frontend — pliki po migracji 2D → 3D:

- static/js/board.js — stara plansza canvas, niepodpięta.
- static/js/board-preview.js, static/js/lootbox.js — brak referencji w szablonach.
- templates/game/room.html — brak kontrolera, zastąpiony przez lobby.html.

Duplikacja źródła prawdy: tablice w GameEconomy.java vs tabele board_tiles/board_cards w PostgreSQL. Zmiana ekonomii w kodzie nie aktualizuje automatycznie słowników w bazie.

5. Struktura bazy danych

Warstwa trwała (schema monopoly, PostgreSQL):

- Użytkownicy: users, player_statistics, owned_items, match_history.
- Społeczność: friendships, profile_comments, game_invites, user_warnings.
- Rozgrywka: game_sessions, game_players + 4 tabele element-collection (posesje, level, landing, karty).
- Słowniki (głównie nieużywane w runtime): board_tiles, board_cards, ranks, daily_tasks, properties.

5.1. Problemy wykorzystania bazy podczas gry

- Każdy rzut kostką, kupno, skip, upgrade = transakcja JPA + `sessionRepository.save()`.
- GamePlayer ma FetchType.EAGER na wszystkich kolekcjach — każdy odczyt sesji ładuje 4 tabele pomocnicze.
- Brak projekcji / EntityGraph — pełny graf obiektów przy każdym GET `/api/game/{id}/state`.
- Polling co 4 s i 5 s w board3d.js (fallback) generuje dodatkowy ruch HTTP + zapytania DB.
- GameStateDto wysyła pełny stan gry przy każdym broadcastzie WebSocket — duży JSON.

6. Architektura: baza danych vs pamięć serwera (RAM)

OBECNY STAN — wszystko w bazie:

Aktywna rozgrywka (pozycje pionków, gotówka, pending kupna/płatności, tura, karty w ręce) jest trzymana w PostgreSQL i zapisywana przy KAŻDEJ akcji gracza. To podejście zapewnia trwałość po restarcie serwera, ale jest kosztowne: setki zapytań SQL na jedną grę, opóźnienia transakcyjne, obciążenie Hibernate i wolniejsza reakcja UI — szczególnie na słabszych urządzeniach i przy wielu równoległych grach.

REKOMENDOWANY STAN — rozgrywka w RAM serwera:

Stan aktywnej gry (status ACTIVE) powinien żyć w pamięci procesu Spring Boot — np. `ConcurrentHashMap<Long, ActiveGameState>` lub Redis (przy wielu instancjach). Operacje roll/buy/skip modyfikują obiekt w RAM (mikrosekundy), a WebSocket od razu pushuje lekki DTO do klientów. Dopiero przy zakończeniu meczu, wyjściu ostatniego gracza lub crash-recovery (opcjonalny snapshot co N tur) następuje zapis do PostgreSQL: MatchHistory, aktualizacja ELO/coins, ewentualnie archiwum sesji.

Proponowany podział odpowiedzialności:

Typ danych	Teraz	Powinno być
Stan ACTIVE (tura, pending, pozycje)	PostgreSQL	RAM serwera (ActiveGameStore)
Lobby WAITING	PostgreSQL	PostgreSQL (OK — mało operacji)
Profil, monety, ELO, inventory	PostgreSQL	PostgreSQL (trwale metadane)
Słownik planszy	DB + kod (	Tylko kod (GameEconomy) lub tylko DB
Draft edycji profilu (PDF)	Brak	HttpSession → flush przy logout

Korzyści przeniesienia rozgrywki do RAM:

- 10–100× mniej zapytań SQL w trakcie meczu.
- Szybsza odpowiedź REST/WebSocket — brak czekania na commit transakcji przy każdym rzucie.
- Mniejsze obciążenie PostgreSQL — baza tylko dla danych trwałych i lobby.
- Lepsza responsywność na słabszych urządzeniach (mniej opóźnień sieciowych po stronie serwera).
- Łatwiejsze anulowanie starych timerów bota (BotAutoplayService) bez race z DB.

Uwaga egzaminacyjna: wymaganie PDF „zapis do bazy dopiero przy wylogowaniu” idealnie pasuje do wzorca: stan roboczy w sesji/RAM → persist przy logout/koniec gry. Obecna implementacja robi odwrotnie (zapis natychmiastowy), co jest sprzeczne z PDF, ale logiczne dla multiplayer — warto to opisać na prezentacji jako świadomą decyzję architektoniczną z planem migracji do ActiveGameStore.

7. Wydajność po stronie klienta (bez rezygnacji z 3D)

Plansza 3D (board3d.js, ~142 KB, ~2790 linii) pozostaje głównym elementem wizualnym — nie proponuje się trybu 2D, aby nie psuć estetyki projektu. Optymalizacje w ramach 3D:

- Wyłączyć polling HTTP gdy WebSocket jest połączony (obecnie 4 s + 5 s niezależnie).
- Delta updates w WebSocket — wysłać tylko zmienione pola, nie pełny GameStateDto.
- Lazy load modeli GLTF pionków; cache w przeglądarce.
- prefers-reduced-motion: skrócone animacje kostki/ruchu (bez zmiany wyglądu planszy).
- Cache busting wersji JS (np. ?v=...) — uniknięcie starego board3d.js w przeglądarce.
- Jeden rebuild mvn clean package po zmianach — IntelliJ często trzyma stary target/.

8. Rekomendacje — priorytety

A. Egzamin (wysoki priorytet):

- Przywrócić PropertyController + widoki CRUD z sortowaniem (3 kryteria) i cookies.
- Dodać filtrowanie listy Property po dacie i colorGroup.
- Draft profilu w HttpSession + zapis User przy logout (wymaganie PDF).
- Serwerowy klient REST (np. RestTemplate → API awatarów przy rejestracji).
- Zaktualizować docs/MAPA-WYMAGAN.md — mapowanie wymagania → plik.

B. Architektura i wydajność (średni priorytet):

- Wprowadzić ActiveGameStore — stan ACTIVE w RAM, flush do DB na FINISHED.
- GamePlayer: LAZY + DTO zamiast EAGER element collections przy odczycie.
- Usunąć lub podłączyć martwe encje (Achievement, GameLog, Property CRUD demo).
- Jedno źródło prawdy planszy — GameEconomy LUB board_tiles, nie oba.
- Naprawić BotAutoplayService — anulowanie starych timerów (bug „Brak aktywnego pola do kupna”).

C. Rozszerzenia gry Monopoly (niski priorytet / po egzaminie):

- Handel nieruchomościami między graczami (obecnie tylko transfer gotówki).
- Czat w pokoju przez WebSocket /topic/game/{id}/chat.
- Tutorial przy pierwszej grze; replay z GameLog (gdy encja zacznie być używana).
- Testy integracyjne GameService (src/test jest pusty).

9. Podsumowanie

Politechnika Monopoly to dojrzały projekt gry wieloosobowej wykraczający poza standard kursowy. Formalnie brakuje kilku punktów z PDF (Property CRUD, sort/filter, zapis sesyjny, klient REST Java). Największy problem techniczny to trzymanie całej aktywnej rozgrywki w PostgreSQL z zapisem przy każdym ruchu — powinno to żyć w pamięci serwera, a baza powinna służyć profilom, historii meczów i lobby. Plansza 3D zostaje bez uproszczeń wizualnych; wydajność poprawia się przez RAM po stronie serwera, mniejsze payloady WebSocket i usunięcie zbędnego pollingu.